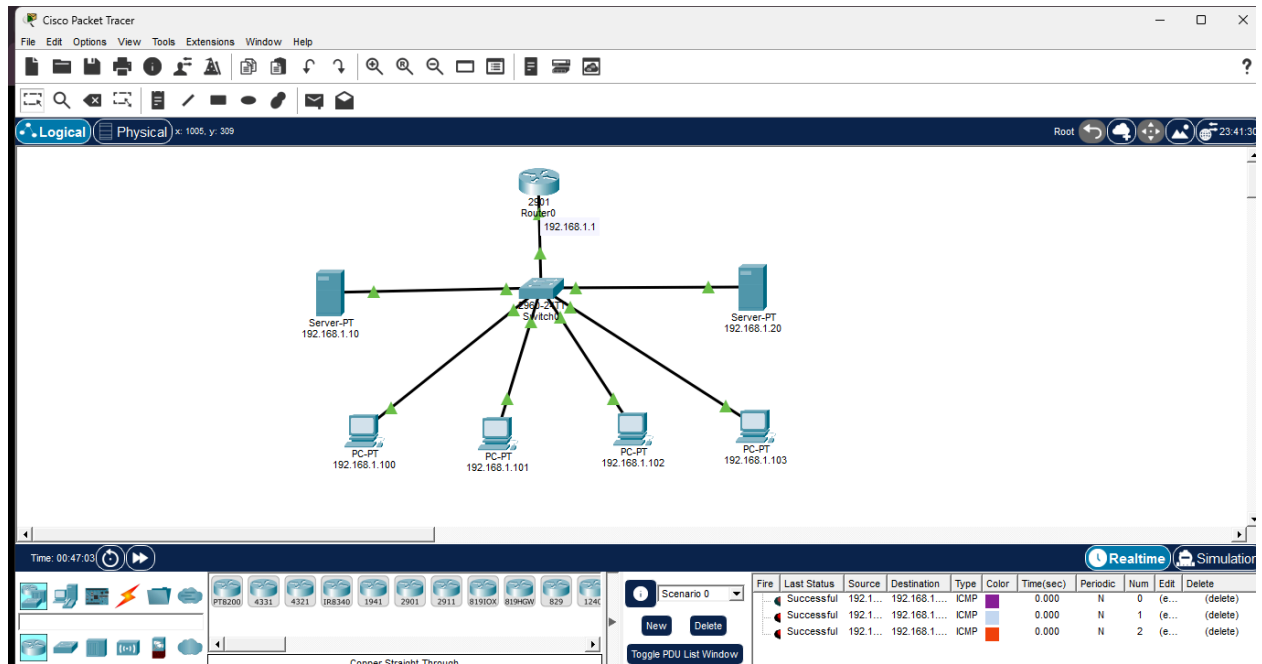
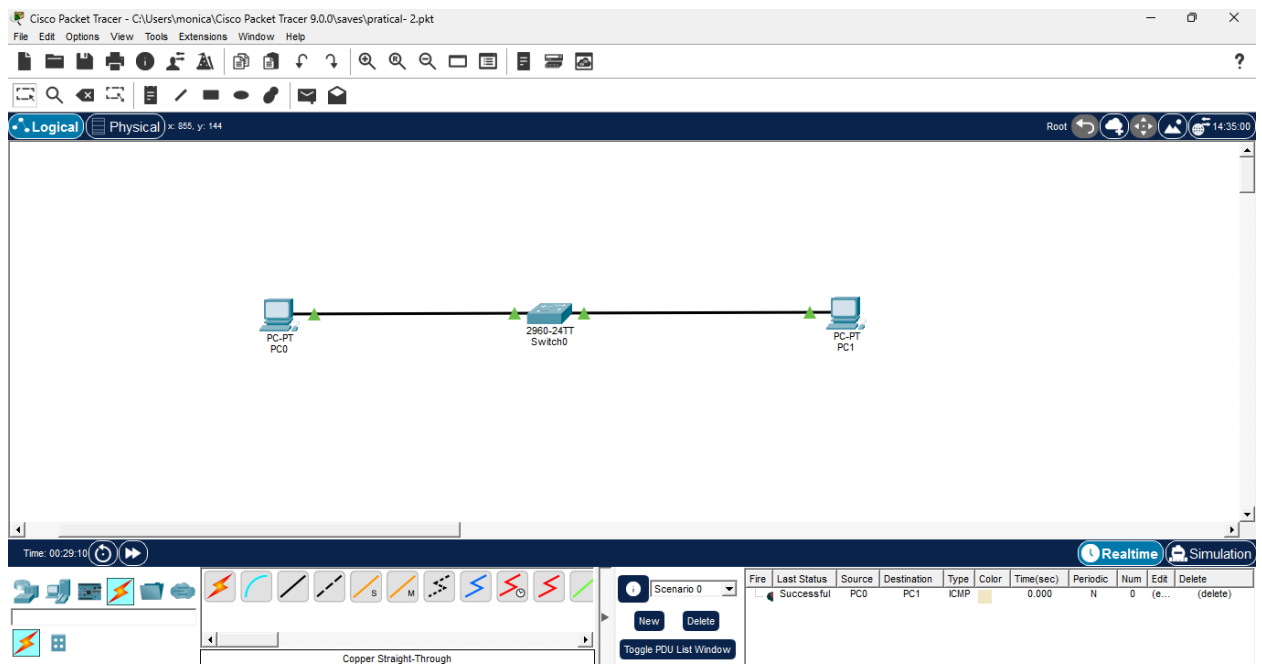


MODEL PRACTICAL EXAM

QUESTION 1:



QUESTION - 2



QUESTION 3:

Capturing from Wi-Fi						
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
icmp						
No.	Time	Source	Destination	Protocol	Length	Info
315	9.731521500	172.25.102.49	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=252/64512, ttl=1 (no response found!)
328	9.896158600	172.25.96.1	172.25.102.49	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
329	9.898400300	172.25.102.49	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=253/64768, ttl=1 (no response found!)
330	9.901978400	172.25.96.1	172.25.102.49	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
331	9.903466400	172.25.102.49	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=254/65024, ttl=1 (no response found!)
332	10.071590000	172.25.96.1	172.25.102.49	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
363	10.749854200	172.25.96.1	172.25.102.49	ICMP	70	Destination unreachable (Port unreachable)
417	12.001862000	172.25.96.1	172.25.102.49	ICMP	70	Destination unreachable (Port unreachable)
520	15.919360600	172.25.102.49	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=255/65280, ttl=2 (no response found!)
521	15.934461600	172.22.10.1	172.25.102.49	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
522	15.936078600	172.25.102.49	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=256/1, ttl=2 (no response found!)
523	16.148750600	172.22.10.1	172.25.102.49	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
524	16.150061800	172.25.102.49	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=257/257, ttl=2 (no response found!)
525	16.169725000	172.22.10.1	172.25.102.49	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
529	16.211782800	172.22.10.1	172.25.102.49	ICMP	70	Destination unreachable (Port unreachable)

> Frame 315: Packet, 106 bytes on wire (848 bits), 106 bytes captured (848 bits) on interface \Device\NPF{...}	0000	24 d5 e4 6d c0 e7 8c 8b 5b 88 4d 29 08 00 45 00	\$-m-...[M]-E:
> Ethernet II, Src: WNC_88:14:d:29 (8c:8b:5b:88:4d:29), Dst: Cisco_6d:c0:e7 (24:d5:e4:6d:c0:e7)	0010	00 5c 21 fb 00 00 01 01 75 4c ac 19 66 31 08 08	..I.....uL..fi..
> Internet Protocol Version 4, Src: 172.25.102.49, Dst: 8.8.8.8	0020	08 08 08 00 f7 02 00 01 00 fc 00 00 00 00 00 00	
> Internet Control Message Protocol	0030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
	0040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
	0050	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
	0060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

```
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

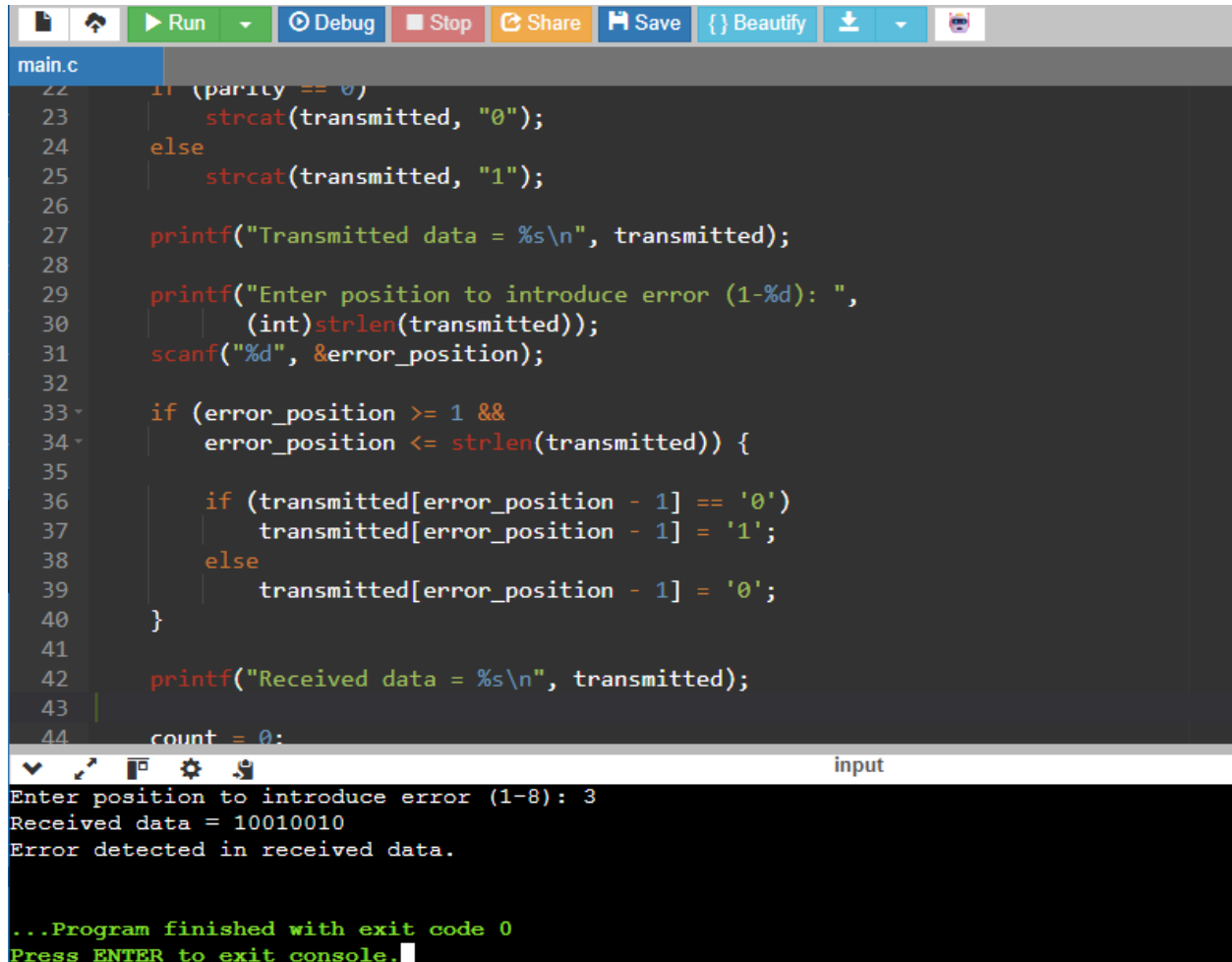
C:\Users\monica>tracert 8.8.8.8

Tracing route to dns.google [8.8.8.8]
over a maximum of 30 hops:
  0  164 ms  3 ms  168 ms  172.25.96.1
  1  15 ms  212 ms  19 ms  172.22.10.1
  2  24 ms  49 ms  6 ms  10.139.187.225
  3  * * * Request timed out.
  4  * * * Request timed out.
  5  * * * Request timed out.
  6  * * * Request timed out.
  7  * * * Request timed out.
  8  86 ms  9 ms  20 ms  72.14.213.20
  9  48 ms  100 ms  26 ms  216.239.43.133
 10  133 ms  37 ms  113 ms  142.251.55.43
 11  246 ms  26 ms  25 ms  dns.google [8.8.8.8]

Trace complete.

C:\Users\monica>
```

QUESTION 4:



The image shows a code editor window with a file named `main.c`. The code is a C program that simulates a parity check. It starts by checking if the parity is 0. If it is, it appends "0" to the `transmitted` string; otherwise, it appends "1". It then prints the transmitted data. Next, it prompts the user to enter a position to introduce an error (1-8). It reads the input and checks if it is within the bounds of the `transmitted` string. If it is, it flips the bit at that position (0 becomes 1, and 1 becomes 0). Finally, it prints the received data and increments a `count` variable.

```
main.c
22  if (parity == 0)
23      strcat(transmitted, "0");
24  else
25      strcat(transmitted, "1");
26
27  printf("Transmitted data = %s\n", transmitted);
28
29  printf("Enter position to introduce error (1-%d): ",
30         (int)strlen(transmitted));
31  scanf("%d", &error_position);
32
33  if (error_position >= 1 &&
34      error_position <= strlen(transmitted)) {
35
36      if (transmitted[error_position - 1] == '0')
37          transmitted[error_position - 1] = '1';
38      else
39          transmitted[error_position - 1] = '0';
40  }
41
42  printf("Received data = %s\n", transmitted);
43
44  count = 0;
```

input

```
Enter position to introduce error (1-8): 3
Received data = 10010010
Error detected in received data.

...Program finished with exit code 0
Press ENTER to exit console.
```

Model Practical Exam:

1. Load Balancing Across Two Servers

Aim:

To design and configure a network with two servers and demonstrate basic load balancing using Cisco Packet Tracer.

Required Materials:

Cisco Packet Tracer, 1 router, 1 switch, 2 servers, PCs, and network cables.

Procedure:

1. Connect the router, switch, two servers, and PCs
2. Assign suitable IP Address to all devices.
3. Configure the router as the default gateway.
4. Configure both servers to handle client requests.
5. Distribute requests between the two servers and test the network.

Result:

The network was successfully configured, and client requests was distributed between the two servers, demonstrating load balancing.

2. Port Security on a Switch :-

Aim:

To configure port security on a switch and prevent unauthorized devices from accessing the network.

Required Network:

Cisco Packet Tracer , 1 switch , 2 PCs ,
and Networking cables.

Procedure:

1. Connect the PCs to the switch .
2. Configure the required switch port as an access port
3. Enable port security and specify the ~~an~~ authorized MAC address
4. Set the violation mode to shutdown .
5. Connect an unauthorized device and observe the port status.

Result:

Port Security was successfully configured. The authorized device was allowed access , while the unauthorized device was blocked .

3) ICMP TTL Exceeded Using Wireshark.

Aim:

To capture and analyze ICMP Time-to-Live (TTL) Exceeded packets using Wireshark.

Required Materials :

Computer / PC , Wireshark , Internet connection , and command prompt.

Procedure:

1. Open Wireshark and start capturing packets from the Wi-Fi interface
2. Open Command Prompt and execute `tracert 8.8.8.8`.
3. Stop the packet capture after the Traceroute is completed.
4. Apply the `icmp` filter in Wireshark.
5. Identify and analyze the Time-to-live exceeded packets.

Result:

ICMP TTL Exceeded packets were successfully captured and analyzed, showing the intermediate routers in the network path.

4) Parity Bit Generation and Error Detection Using C.

Aim:

To develop a C program for generating an even parity bit and detecting errors in transmitted binary data.

Required Materials:

Computer, VS code (or) C compiler, and C programming environment.

Procedure:

1. Enter the binary data as input.
2. Count the number of 1s and generate the even parity bit.
3. Append the parity bit to the data.
4. Introduce an error by changing one bit.
5. Check the received data and determine whether an error has occurred.

Result:

The C program successfully generated the parity bit and detected the error in the received binary data.